

INFORME TAREA 1 - DESARROLLO DE SOFTWARE

ERICK DANIEL ORTEGA MORAN - 20210209H

Link al repositorio GitHub: https://github.com/sammyboy100/T1-Desarrollo-de-Software-Erick_Ortega_20210209H.git

Pregunta 1: Simulador de Enrutamiento Web

Descripción del problema:

Se requiere implementar un simulador de enrutamiento para una aplicación web de una sola página (SPA). El programa debe recibir un conjunto de rutas válidas (algunas con parámetros dinámicos) y evaluar las peticiones ingresadas por el usuario, determinando el contenido a mostrar o devolviendo un error 404 Not Found si la ruta es inexistente.

Explicación de la solución:

En primer lugar, el programa utiliza la función `open()` para leer los datos estructurados directamente desde el archivo de texto (`input.txt`). Tras limpiar los espacios y saltos de línea innecesarios, las rutas base se almacenan en una lista de tuplas. Un detalle crucial en el desarrollo fue el uso de la función `split(maxsplit=1)`. Esto permite separar correctamente el "path" de su contenido asociado, evitando errores si dicho contenido posee espacios intermedios (por ejemplo: `UserPage {id}`).

Luego, se evalúa cada transición (petición) del usuario separándola por el carácter `/`. El código compara los bloques de la petición con los de la ruta utilizando la función `zip()`. Si se identifica que un bloque de la ruta base empieza con dos puntos (`:`), lo reconoce inmediatamente como un parámetro dinámico y almacena el valor.

Finalmente, si la ruta coincide, el programa emplea la librería `re` (expresiones regulares). Mediante la instrucción `re.sub(r'\{.*?\}', '', contenido)`, el algoritmo elimina cualquier corchete en el texto base, logrando imprimir una salida limpia concatenada con el parámetro capturado. En caso de no existir ninguna coincidencia, se imprime `404 Not Found`.

Código fuente (p1DS.py):

```
1 import re
2
3 def simular_enrutador(archivo_entrada):
4     # Abrimos y leemos el archivo input.txt
5     with open(archivo_entrada, 'r') as archivo:
6         lineas = [linea.strip() for linea in archivo.readlines() if
7                     linea.strip()]
```

```

8     if not lineas:
9         return
10
11     N = int(lineas[0])
12     rutas = []
13
14     for i in range(1, N + 1):
15         partes = lineas[i].split(maxsplit=1)
16         path = partes[0]
17         contenido = partes[1] if len(partes) > 1 else ""
18         rutas.append((path, contenido))
19
20     indice_M = N + 1
21     M = int(lineas[indice_M])
22     transiciones = lineas[indice_M + 1 : indice_M + 1 + M]
23
24     for transicion in transiciones:
25         t_partes = transicion.split('/')
26         encontrado = False
27
28         for path, contenido in rutas:
29             r_partes = path.split('/')
30
31             if len(t_partes) == len(r_partes):
32                 coincide = True
33                 parametros = []
34
35                 for r, t in zip(r_partes, t_partes):
36                     if r.startswith(':'):
37                         parametros.append(t)
38                     elif r != t:
39                         coincide = False
40                         break
41
42                 if coincide:
43                     encontrado = True
44                     contenido_limpio = re.sub(r'\{.*?\}', '', contenido)
45
46                     if parametros:
47                         print(f"{contenido_limpio} {' '.join(parametros)}")
48
49                     else:
50                         print(contenido_limpio)
51                         break
52
53     if not encontrado:

```

```
53         print("404 Not Found")
54
55 if __name__ == '__main__':
56     simular_enrutador('input.txt')
```

Salida de la terminal:

```
HomePage
ProfilePage
UserPage 42
404 Not Found
```

Pregunta 2: Cliente Más Fiel por Socio

Descripción del problema:

El Banco de la Nación desea analizar un historial de transacciones masivas para determinar qué cliente ha realizado la mayor cantidad de compras en los terminales (POS) de cada uno de sus socios. En caso de empate en la cantidad de compras, se debe seleccionar al cliente con el identificador numérico más pequeño.

Explicación de la solución:

Para lograr una ejecución temporal óptima, el algoritmo basa su estructura en diccionarios de Python (tablas hash) que permiten realizar búsquedas en tiempo constante $O(1)$. Primero, se construye el diccionario `terminal_a_socio`, que relaciona cada identificador de máquina (terminal) con el socio al que le pertenece, evitando ciclos anidados futuros.

Durante la fase de transacciones, se emplea un diccionario anidado `compras_por_socio`. Por cada compra leída en el archivo `input2.txt`, el programa ubica al socio y le suma una operación al cliente. El algoritmo valida explícitamente mediante una condicional si el cliente existe en el historial del socio; de no ser así, lo inicializa con un valor de 0 antes de efectuar la suma.

Una vez procesadas todas las compras, el algoritmo itera sobre la lista de socios. La evaluación y el criterio de desempate se aplican de forma simultánea en la siguiente condición:

```
if cantidad_compras > max_compras or (cantidad_compras == max_compras and cliente_id < mejor_cliente):
```

Esto asegura que, ante una igualdad de compras, se actualice al cliente más fiel eligiendo estrictamente al de menor ID. Si el diccionario del socio se encuentra vacío, el programa arroja como resultado su ID seguido de -1.

Código fuente (p2DS.py):

```
1 def encontrar_clientes_fieles(archivo_entrada):
2     # Leemos el archivo input2.txt
3     with open(archivo_entrada, 'r') as archivo:
4         lineas = [linea.strip() for linea in archivo.readlines() if
5                     linea.strip()]
6
7     if not lineas:
8         return
9
10    valores_principales = lineas[0].split()
11    N = int(valores_principales[0])
12    M = int(valores_principales[1])
13    S = int(valores_principales[2])
14
15    terminal_a_socio = {}
16    compras_por_socio = {i: {} for i in range(1, N + 1)}
```

```

16
17     for i in range(1, M + 1):
18         partes = lineas[i].split()
19         p = int(partes[0])
20         t = int(partes[1])
21         terminal_a_socio[t] = p
22
23     for i in range(M + 1, M + 1 + S):
24         partes = lineas[i].split()
25         c = int(partes[0])
26         t = int(partes[1])
27
28         if t in terminal_a_socio:
29             p = terminal_a_socio[t]
30
31             if c not in compras_por_socio[p]:
32                 compras_por_socio[p][c] = 0
33                 compras_por_socio[p][c] += 1
34
35     for p in range(1, N + 1):
36         clientes_de_este_socio = compras_por_socio[p]
37
38         if not clientes_de_este_socio:
39             print(f"{p} -1")
40         else:
41             mejor_cliente = -1
42             max_compras = -1
43
44             for cliente_id, cantidad_compras in clientes_de_este_socio.
45 items():
46                 if cantidad_compras > max_compras or (cantidad_compras
47 == max_compras and cliente_id < mejor_cliente):
48                     max_compras = cantidad_compras
49                     mejor_cliente = cliente_id
50
51             print(f"{p} {mejor_cliente}")
52
53 if __name__ == '__main__':
54     encontrar_clientes_fieles('input2.txt')

```

Salida de la terminal:

```

1 1501
2 1501
3 1502
4 -1

```